Chapter 3

The Processor

Tracing with Control Signals

1. Instruction

- Identify the instruction type (R-type, `lw`, `sw`, `beq`, `j`).
- Determine the operands and immediate value.
- Read the relevant values from registers / memory / PC.

Example:

```
lw  $t0, 8($t1)
```

2. Control Signals

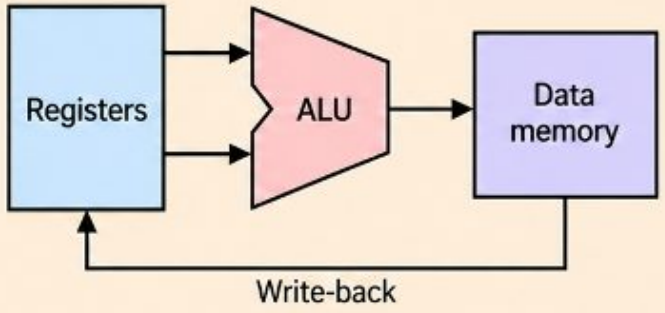
- Use the opcode (and funct for R-type) to determine the control signal values.
- These signals specify how each datapath component should operate.

Example: lw

RegDst	0	MemRead	1
ALUSrc	1	MemWrite	0
MemtoReg	1	Branch	0
RegWrite	1	ALUOp	00

3. Datapath

- Follow the control signals to determine the selected path in the datapath.
- Compute the intermediate values (e.g., ALU result, memory address, data read/write).
- Determine the destination register and the next PC.



4. Execution Result

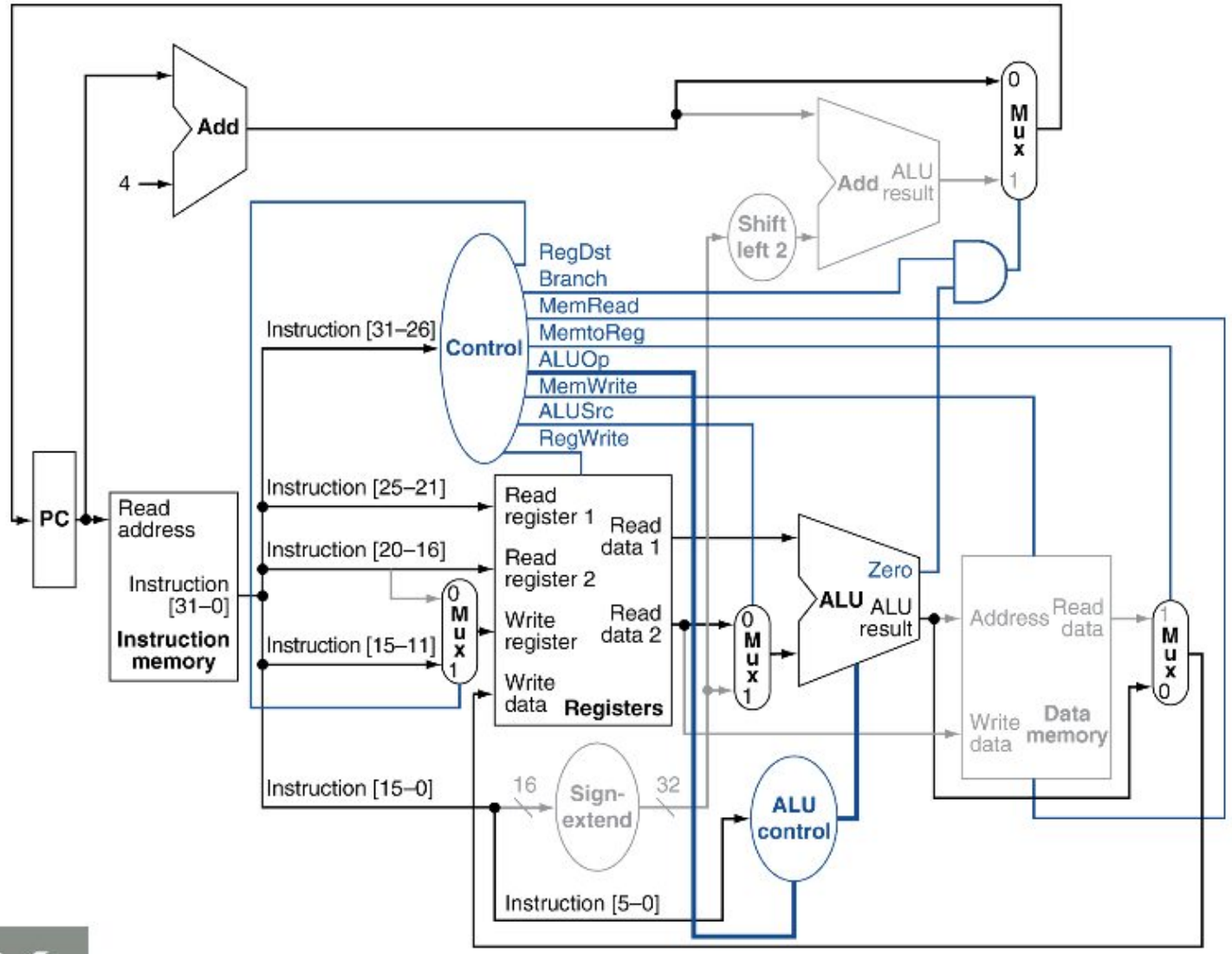
ALU result
- Operation performed by the ALU
- Result value

Memory access
- Read or write?
- Memory address
- Data read or written (if any)

Register write-back
- Destination register
- Value written back (if any)

Next PC
- PC + 4 (normal)
- Branch target (if beq taken)
- Jump target (if j)

R-Type Instruction



R-Format Inst. (Recap)

Example Instruction

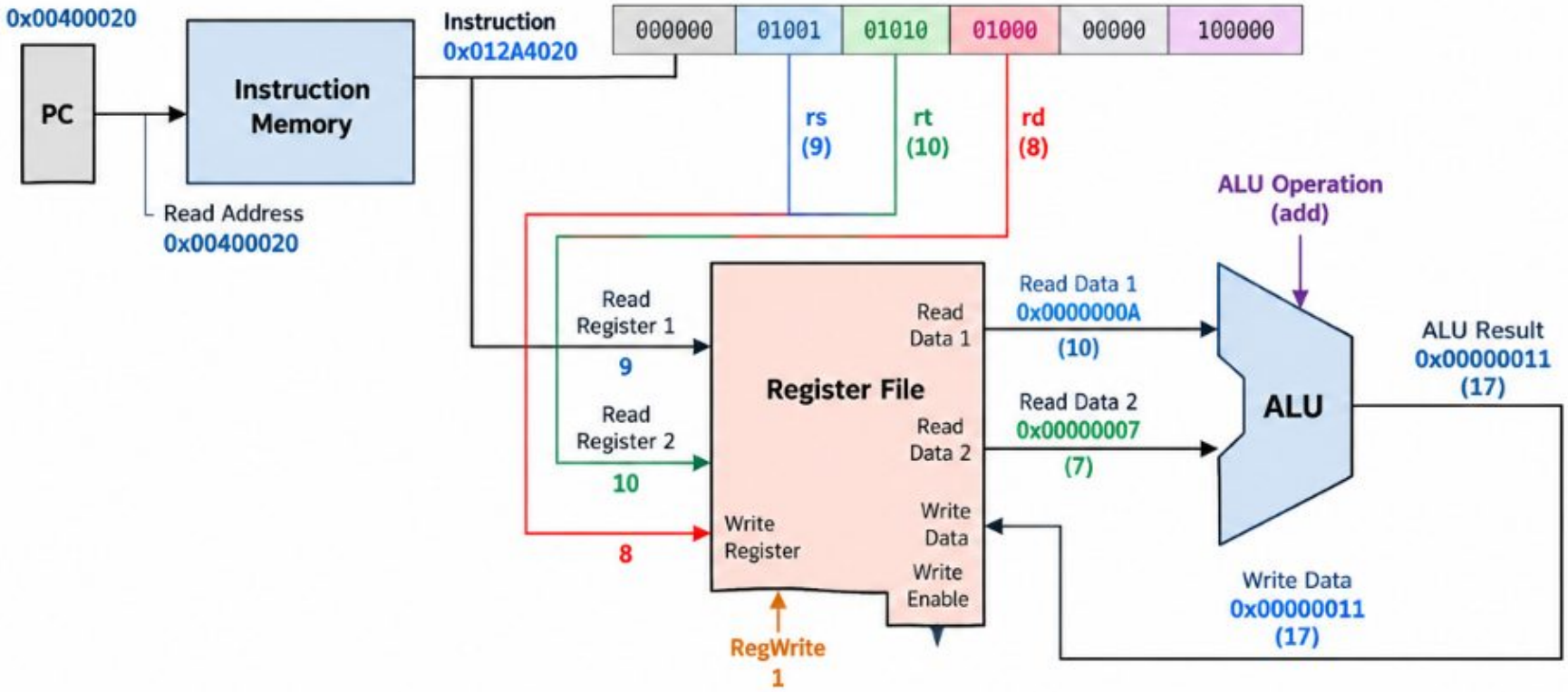
```
add  $t0, $t1, $t2
$t0 = $t1 + $t2
```

Given Register Values

```
$t1 ($9)  = 10 (0x0000000A)
$t2 ($10) = 7  (0x00000007)
```

Instruction Encoding (R-format)

opcode	rs	rt	rd	shamt	funct
000000	01001	01010	01000	00000	100000
0x00	$9 ($t1)	$10 ($t2)	$8 ($t0)	0	0x20 (add)



Controls

Example Instruction

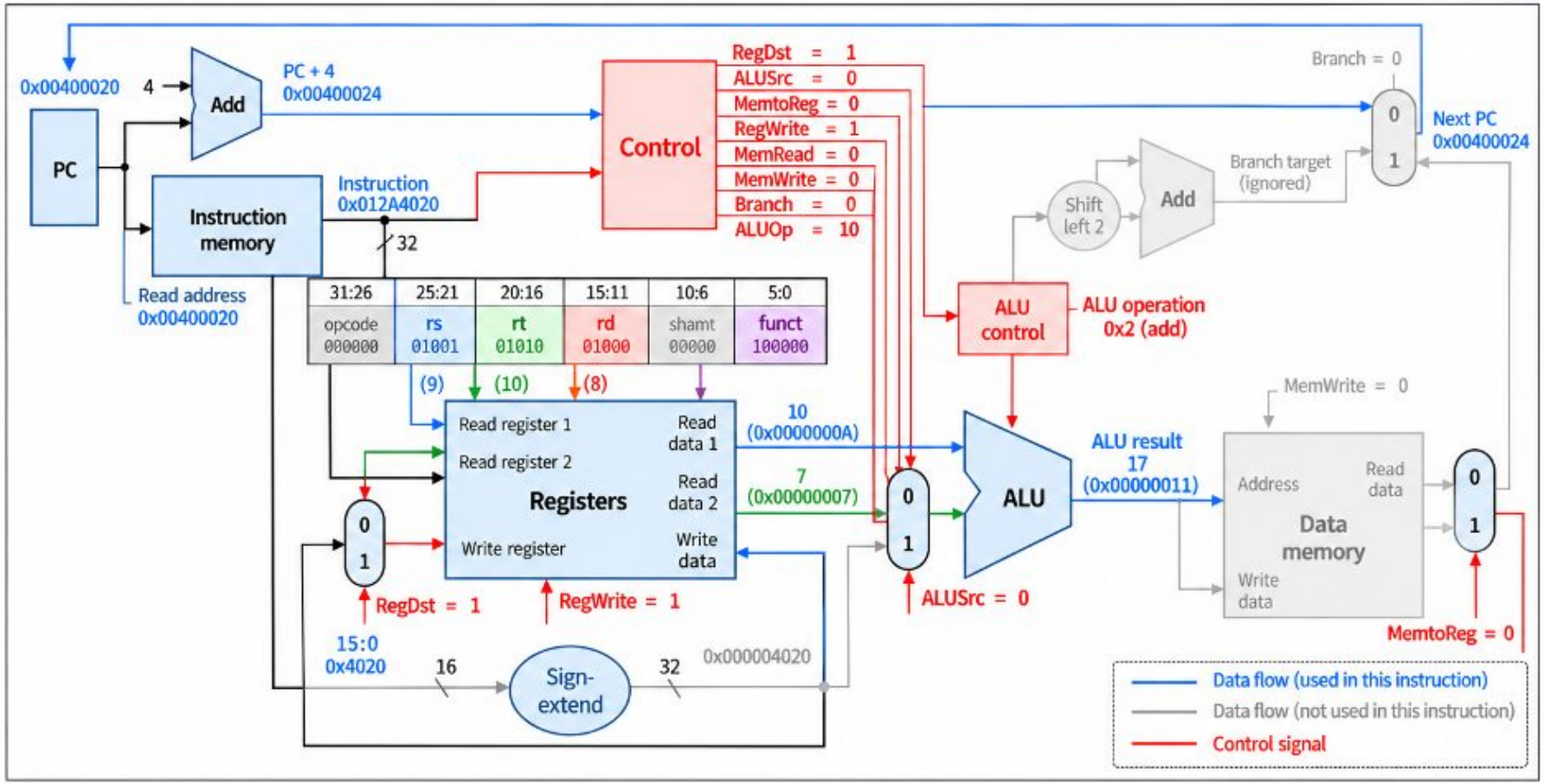
```
add  $t0, $t1, $t2
$t0 = $t1 + $t2
```

Given Register Values

```
$t1 ($9)  =  10  (0x0000000A)
$t2 ($10) =  7   (0x00000007)
$t0 ($8)  =  0   (before)
```

Instruction Encoding (R-format)

opcode	rs	rt	rd	shamt	funct
000000	01001	01010	01000	00000	100000
0x00	**$9** ($t1)	**$10** ($t2)	**$8** ($t0)	0	0x20 (add)



Controls in R-Format Inst.

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	1	Selects rd ($t0) as the destination register
ALUSrc	1 bit	0	Selects Read Data 2 ($t2) as the ALU input
MemtoReg	1 bit	0	Selects ALU Result for write-back
RegWrite	1 bit	1	Enables writing the result to $t0
MemRead	1 bit	0	Disables Data Memory read
MemWrite	1 bit	0	Disables Data Memory write
Branch	1 bit	0	Selects PC + 4 as the next PC
ALUOp	2 bits	10	Uses funct to determine the ALU operation
ALU Control	4 bits	0010	Performs ADD (10 + 7 = 17)

Controls in R-Format Inst.

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	1	Selects rd ($t0) as the destination register
ALUSrc	1 bit	0	Selects Read Data 2 ($t2) as the ALU input
MemtoReg	1 bit	0	Selects ALU Result for write-back

Q1. Consider the following instruction: sub $t0, $t1, $t2

Given $t1 = 20 and $t2 = 8, determine:

The values of RegDst, ALUSrc, MemtoReg, and RegWrite.

(b) The values of ALUOp and ALU Control.

(c) The final value written into $t0.

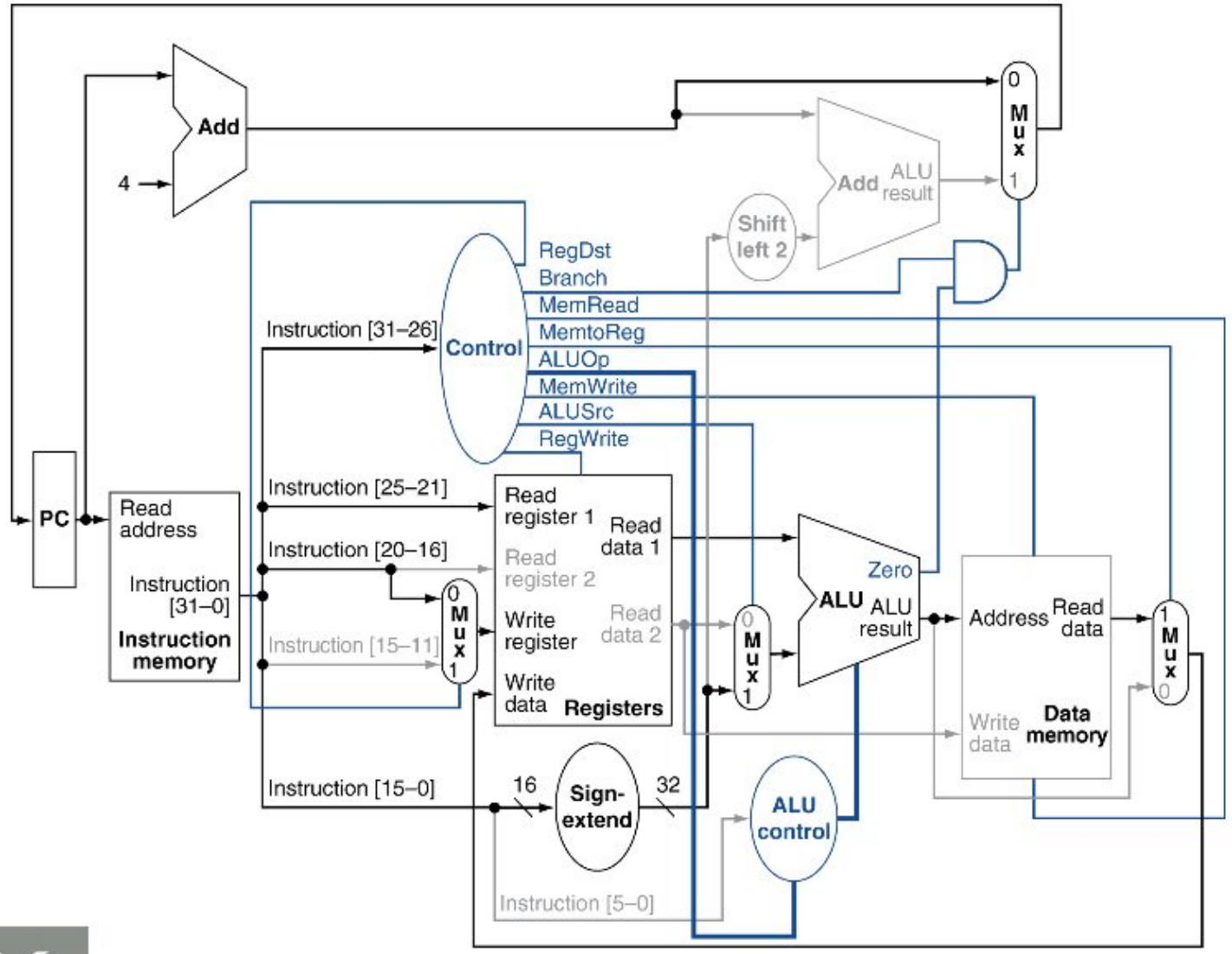
A1.

(a) RegDst = 1, ALUSrc = 0, MemtoReg = 0, RegWrite = 1.

(b) ALUOp = 10, ALU Control = 0110 (SUB).

(c) The ALU performs 20 – 8 = 12. The result is written into

Load Instruction



Load Instruction Controls

Example Instruction

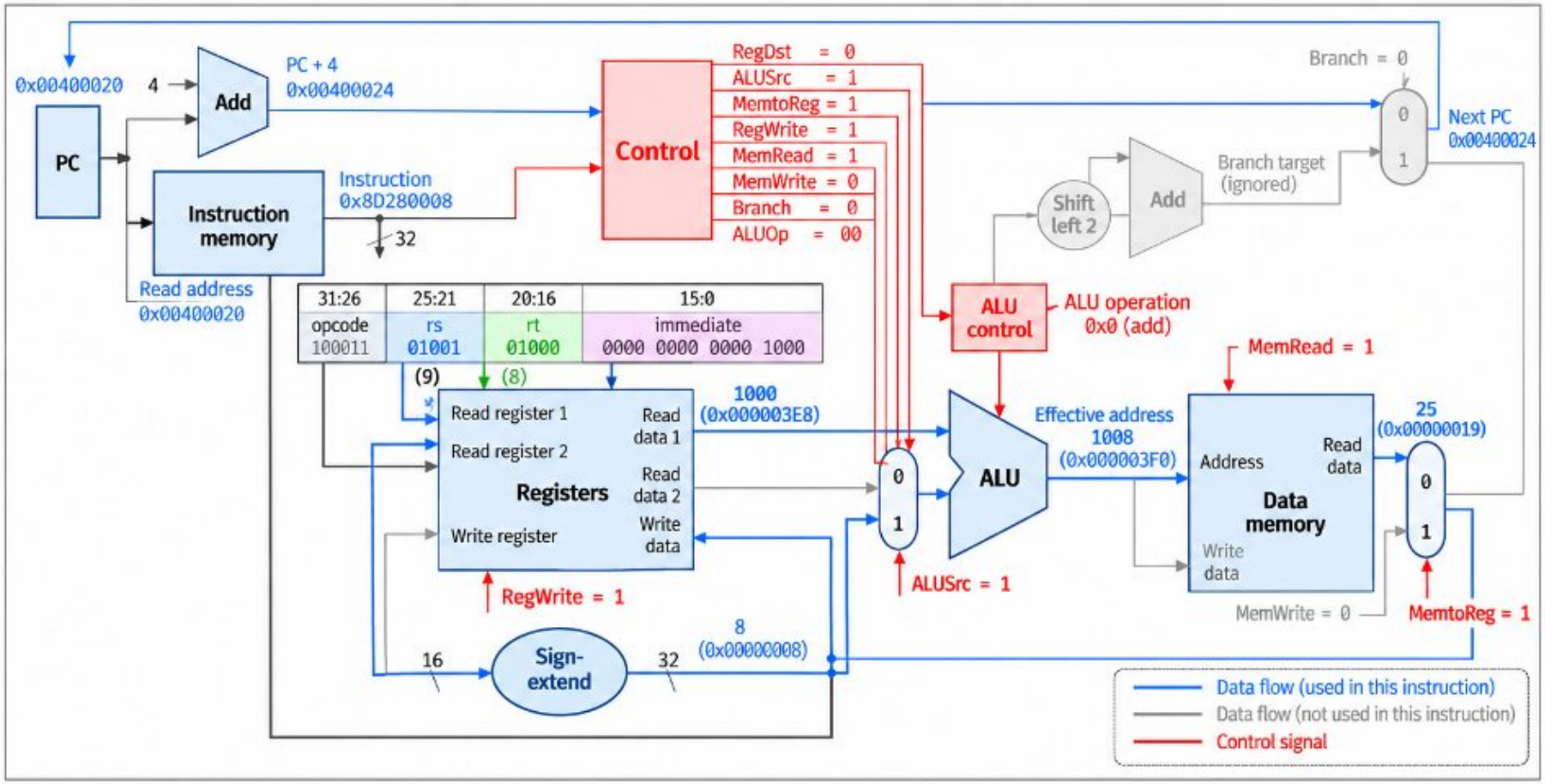
```
lw  $t0, 8($t1)
$t0 = Mem[$t1 + 8]
```

Given Values

PC = 0x00400020
$t1 ($9) = 1000 (0x000003E8)
Memory[1008] = 25 (0x00000019)
$t0 ($8) = 0 (before)

Instruction Encoding (I-format)

opcode	rs	rt	immediate (16 bits)
100011	01001	01000	0000 0000 0000 1000
0x23 (lw)	$9 ($t1)	$8 ($t0)	8 (0x0008)



Controls in Load Instructions

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	0	Selects rt ($t0) as the destination register
ALUSrc	1 bit	1	Selects sign-extended immediate (8) as ALU input
MemtoReg	1 bit	1	Selects Data Memory output for write-back
RegWrite	1 bit	1	Enables writing the loaded value to $t0
MemRead	1 bit	1	Enables Data Memory read
MemWrite	1 bit	0	Disables Data Memory write
Branch	1 bit	0	Selects PC + 4 as the next PC
ALUOp	2 bits	00	Specifies ADD for address calculation
ALU Control	4 bits	0010	Performs ADD (1000 + 8 = 1008)

Controls in Load Instructions

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	0	Selects rt ($t0) as the destination register
ALUSrc	1 bit	1	Selects sign-extended immediate (8) as ALU input
MemtoReg	1 bit	1	Selects Data Memory output for write-back

Q2. Consider the instruction lw $t0, 8($t1)

Which control signals must be set to 1 to calculate the effective address, read data from memory, and write the loaded value into $t0?

A2.

ALUSrc = 1, MemRead = 1, MemtoReg = 1, and RegWrite = 1.

ALUSrc selects the immediate offset for address calculation, MemRead enables memory reading, MemtoReg selects the memory data, and RegWrite enables writing the loaded value into $t0.

Store Instruction Controls

Example Instruction

```
sw  $t2, 8($t0)
```

PC = 0x00400020 → 0x00400024

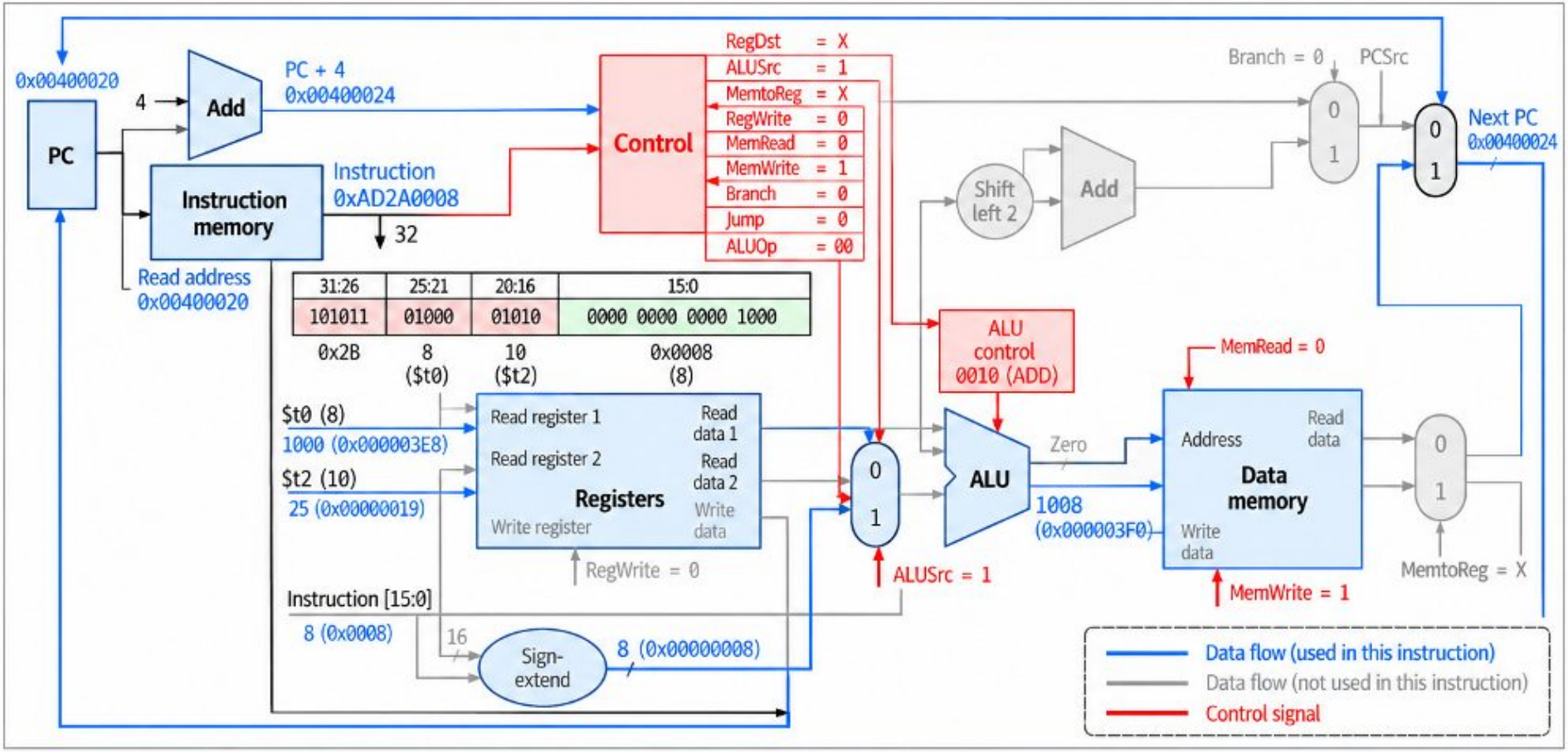
Given Values

PC	= 0x00400020	
PC + 4	= 0x00400024	
$t0	= 1000	(0x000003E8)
$t2	= 25	(0x00000019)
Immediate	= 8	(0x0008)
Effective address	= 1008	(0x000003F0)

Instruction Encoding (I-format)

opcode (6)	rs (5)	rt (5)	immediate (16)
101011	01000	01010	0000 0000 0000 1000
0x2B (sw)	8 ($t0)	10 ($t2)	0x0008 (8)

Instruction (hex) = **0xAD2A0008**



Controls in Store Instructions

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	1	Selects sign-extended immediate (8) as ALU input
MemtoReg	1 bit	X	Don't care (no register write-back)
RegWrite	1 bit	0	Disables Register File write
MemRead	1 bit	0	Disables Data Memory read
MemWrite	1 bit	1	Enables writing $t2 (25) to Data Memory
Branch	1 bit	0	Selects PC + 4 as the next PC
ALUOp	2 bits	00	Specifies ADD for address calculation
ALU Control	4 bits	0010	Performs ADD (1000 + 8 = 1008)

Controls in Store Instructions

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	1	Selects sign-extended immediate (8) as ALU input

Q3. A MIPS processor executes an instruction with the following control signals:

RegWrite = 0, MemRead = 0, MemWrite = 1, ALUSrc = 1, ALUOp = 00

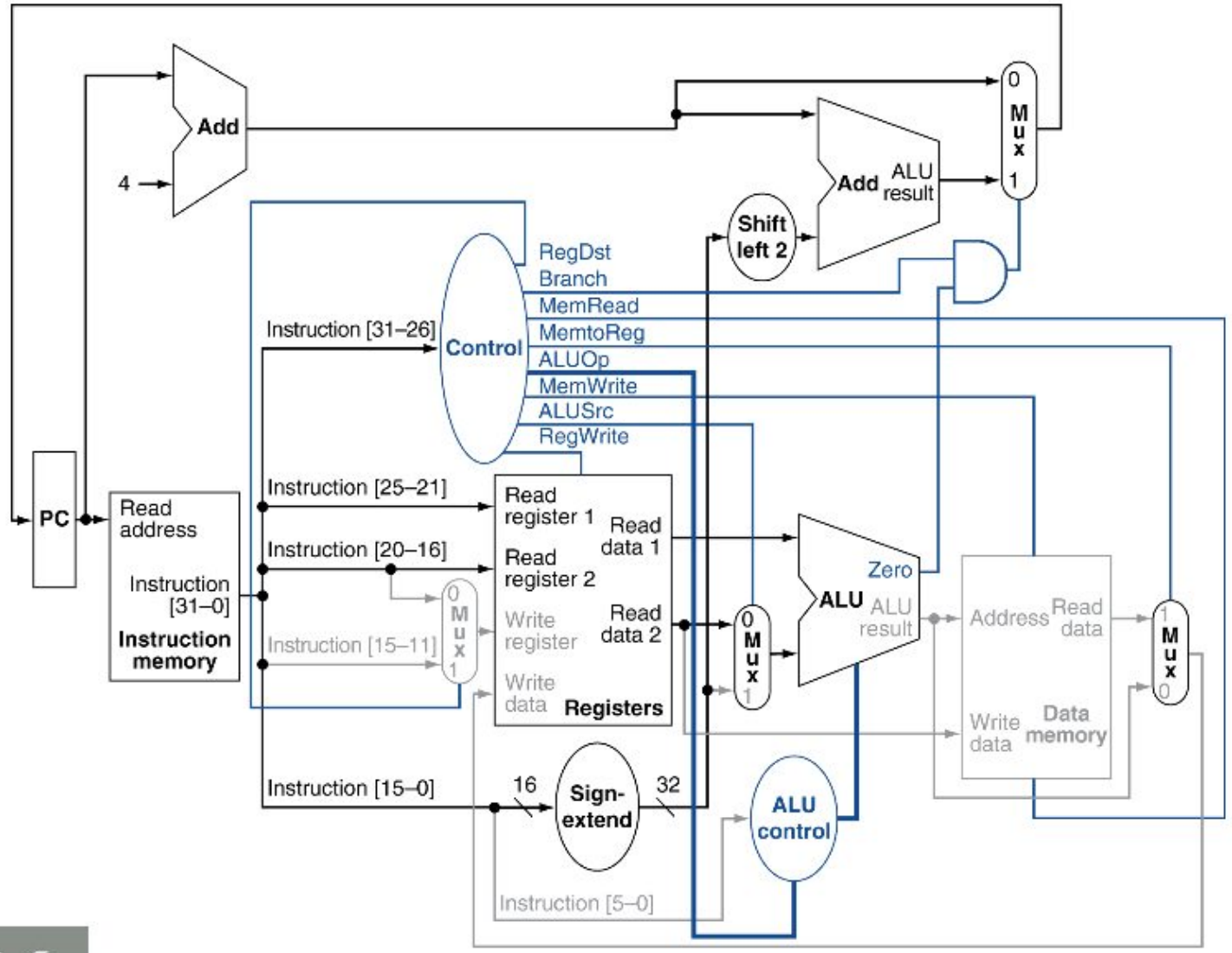
What type of instruction is being executed, and how does the processor use the ALU result?

A3.

The processor is executing a Store (sw) instruction.

The ALU adds the base register value and the sign-extended immediate offset to calculate the effective memory address. The processor then writes the second register's data into that memory address.

Branch-on-Equal Instruction



Branch Instruction Controls

Example Instruction

beq $t0, $t2, 12

PC = 0x00400020 → 0x00400054

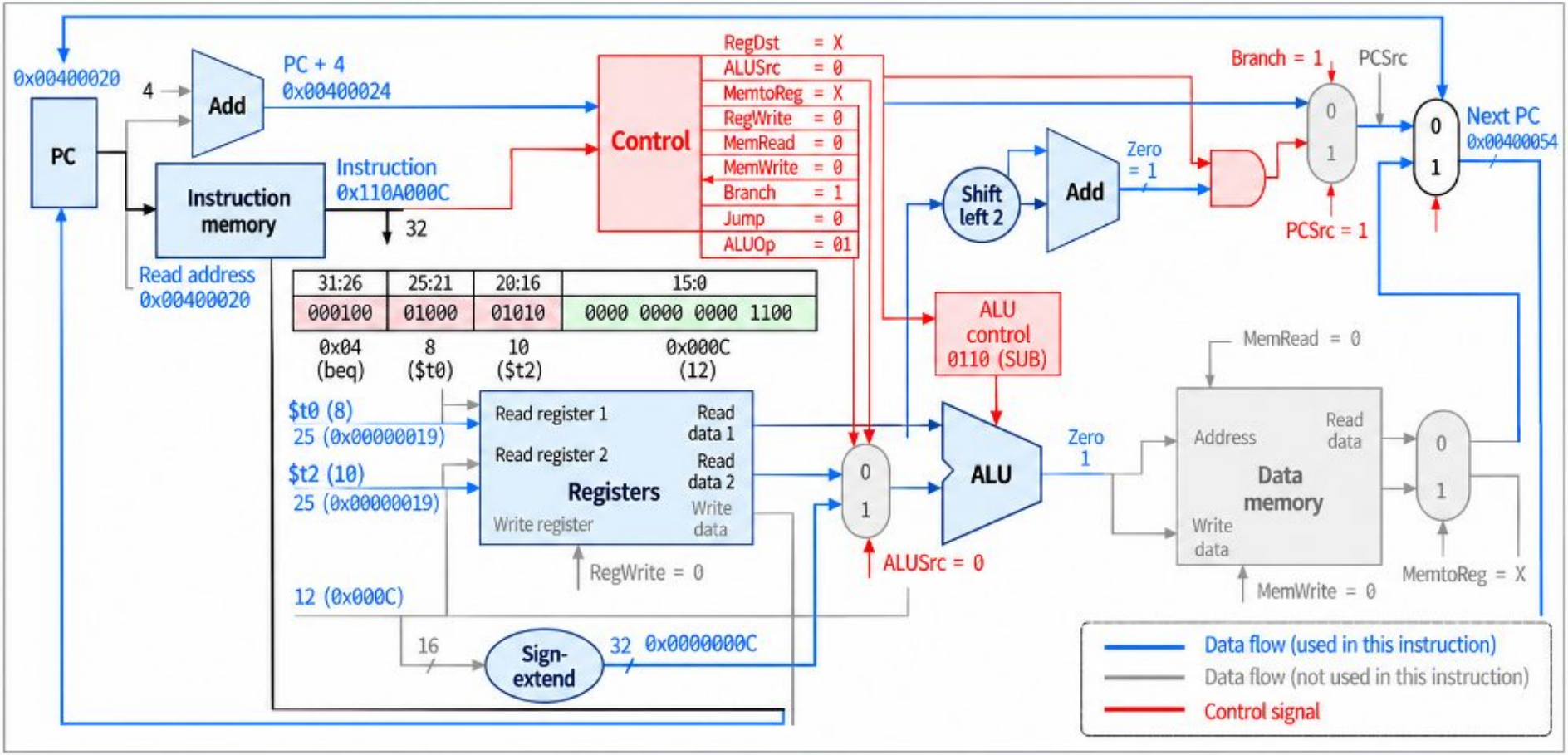
Given Values

PC	= 0x00400020	
PC + 4	= 0x00400024	
$t0	= 25	(0x00000019)
$t2	= 25	(0x00000019)
Immediate	= 12	(0x000C)
Branch target	= 0x00400054	

Instruction Encoding (I-format)

opcode (6)	rs (5)	rt (5)	immediate (16)
000100	01000	01010	0000 0000 0000 1100
0x04 (beq)	8 ($t0)	10 ($t2)	0x000C (12)

Instruction (hex) = **0x110A000C**



Instruction

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	0	Selects Read Data 2 ($t2) as the ALU input
MemtoReg	1 bit	X	Don't care (no register write-back)
RegWrite	1 bit	0	Disables Register File write
MemRead	1 bit	0	Disables Data Memory read
MemWrite	1 bit	0	Disables Data Memory write
Branch	1 bit	1	Enables conditional branch decision
ALUOp	2 bits	01	Specifies SUBTRACT for register comparison
ALU Control	4 bits	0110	Performs SUBTRACT (25 − 25 = 0)
Zero	1 bit	1	Indicates the two register values are equal
PCSrc	1 bit	1	Selects branch target (0x00400054)

Instruction

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	0	Selects Read Data 2 ($t2) as the ALU input
MemtoReg	1 bit	X	Don't care (no register write-back)
RegWrite	1 bit	0	Disables Register File write

A4. Consider the instruction beq $t1, $t2, 12, where $t1 = 25, $t2 = 25, and PC = 0x00400020. Given Branch = 1, determine the values of Zero and PCSrc, and calculate the Next PC.

A4.

Zero = 1 because the two register values are equal.

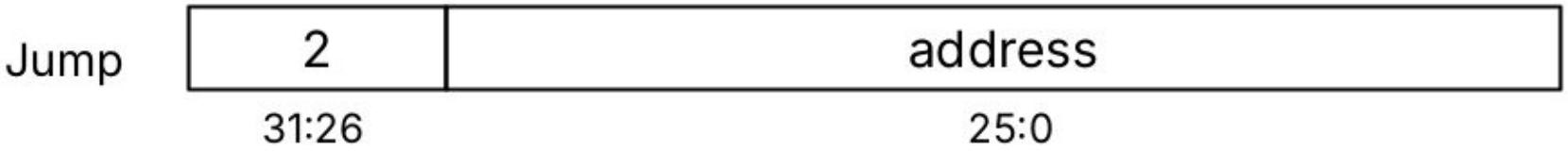
PCSrc = Branch AND Zero = 1 AND 1 = 1.

PC + 4 = 0x00400024.

Branch Offset = 12 × 4 = 48 = 0x30.

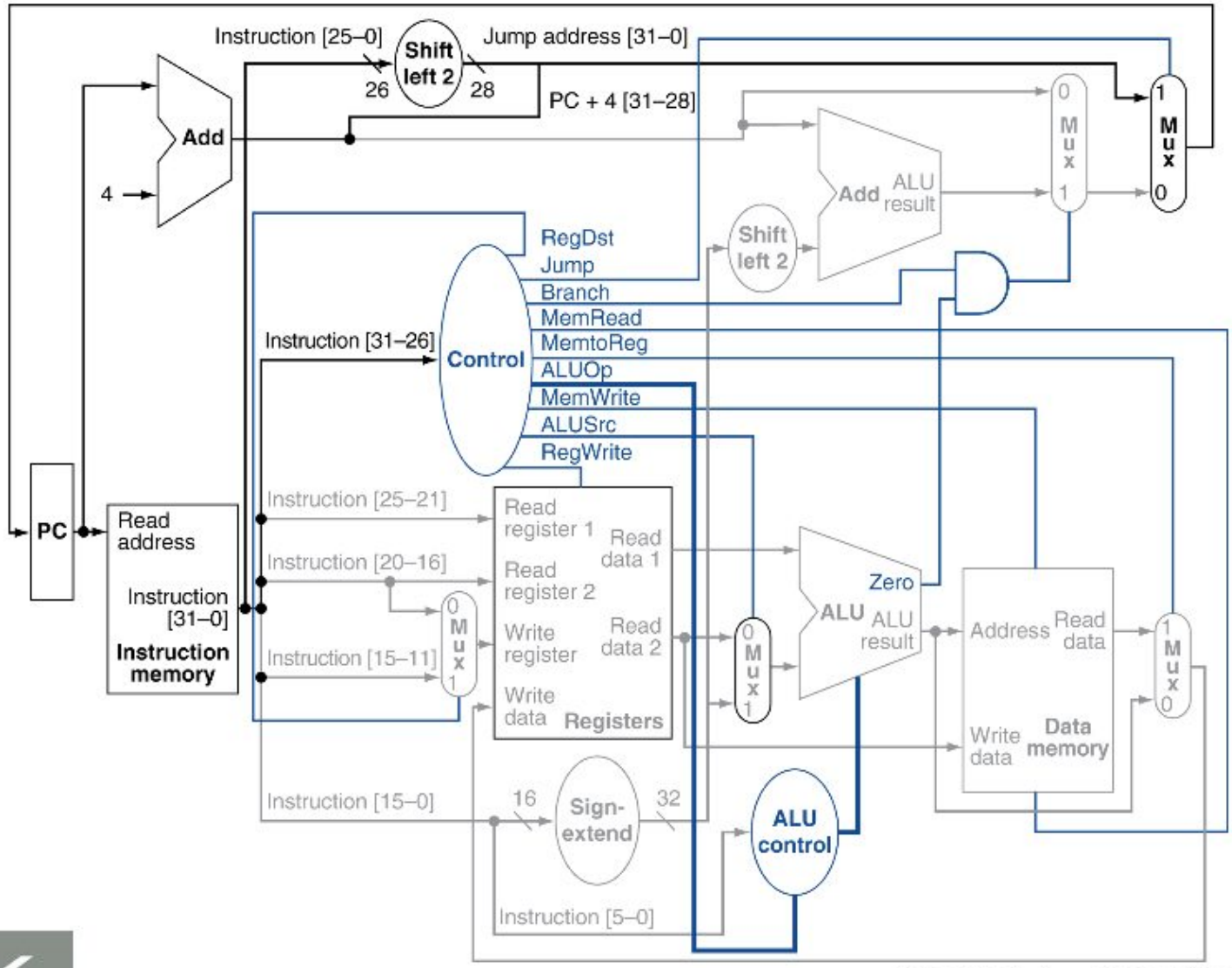
Branch Target = 0x00400024 + 0x30 = 0x00400054.

Implementing Jumps

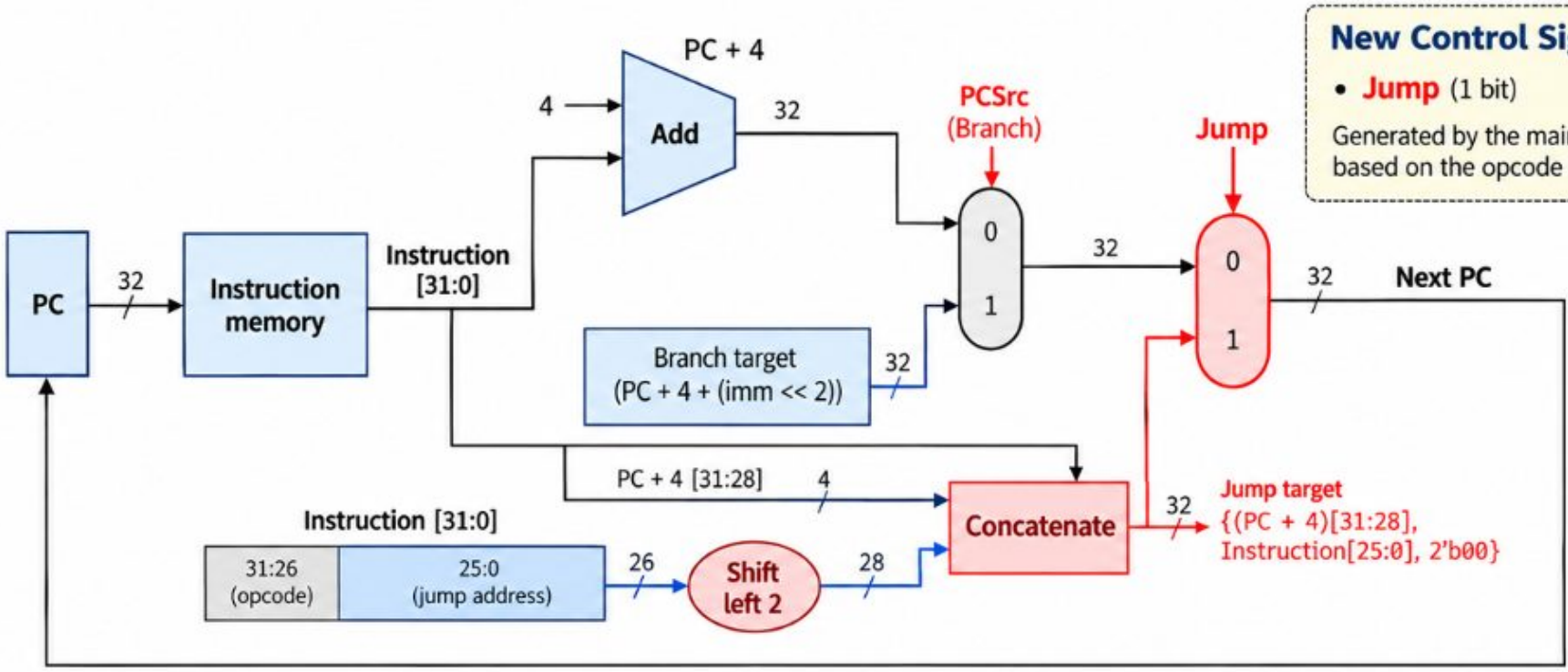


- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of (PC + 4)
  - 26-bit jump address
  - 00
- Need an extra control signal decoded from opcode

Datapath With Jumps Added



Implementing Jump Control



1. Normal Instruction
- Jump = 0
- PCSrc = 0 (or branch not taken)
- Next PC = PC + 4

2. Branch Instruction (beq)
- Jump = 0
- PCSrc = 1 (and Zero = 1)
- Next PC = Branch target

3. Jump Instruction (j)
- Jump = 1
- Next PC = Jump target
  {(PC + 4)[31:28], Instruction[25:0], 2'b00}

Jump Instruction Controls

Example Instruction

```
j   0x00100020
```

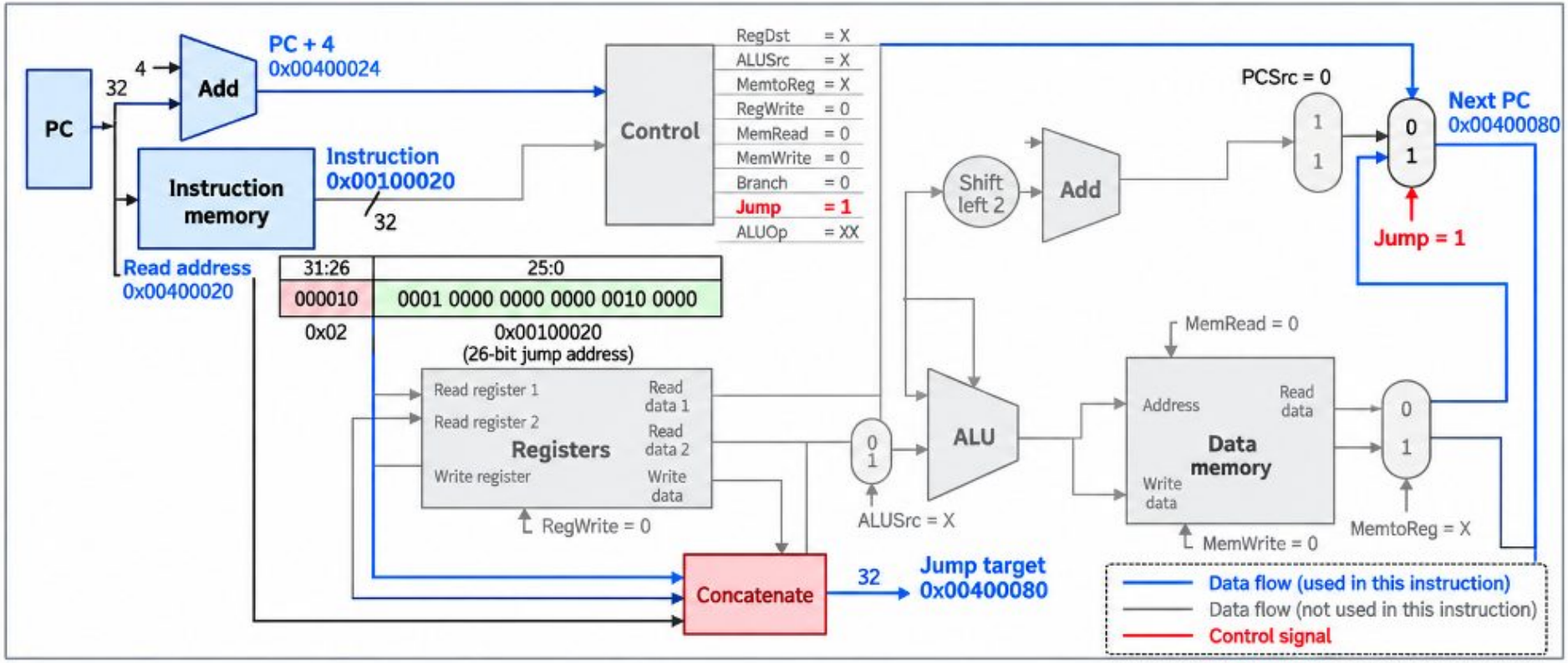
PC = 0x00400020 → 0x00400080

Given Values

PC	= 0x00400020	
Instruction	= 0x00100020	(j)
Jump address	= 0x00100020	
Jump target	= 0x00400080	

Instruction Encoding (J-format)

opcode (6)	address (26)
000010	0001 0000 0000 0000 0010 0000
0x02 (j)	0x00100020 (26-bit jump address)



Controls in Jump Instruction

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	X	Don't care (ALU result is unused)
MemtoReg	1 bit	X	Don't care (no register write-back)
RegWrite	1 bit	0	Disables Register File write
MemRead	1 bit	0	Disables Data Memory read
MemWrite	1 bit	0	Disables Data Memory write
Branch	1 bit	0	Disables conditional branching
Jump	1 bit	1	Selects Jump Target as the next PC
ALUOp	2 bits	XX	Don't care (ALU operation is unused)
ALU Control	4 bits	XXXX	Don't care (ALU result is unused)
PCSrc	1 bit	0	Selects PC + 4 at the Branch MUX; overridden by Jump MUX

Controls in Jump Instruction

Control Signal	Width	Value	Datapath Role
RegDst	1 bit	X	Don't care (no register write-back)
ALUSrc	1 bit	X	Don't care (ALU result is unused)
MemtoReg	1 bit	X	Don't care (no register write-back)

Q5. A processor executes a Jump instruction with Jump = 1,

Branch = 0, and Zero = 0.

If PC + 4 = 0x00400024 and the Jump Target is 0x00400080, determine the value of PCSrc and explain which address becomes the Next PC.

A5.

PCSrc = Branch AND Zero = 0 AND 0 = 0

The Branch MUX selects PC + 4 = 0x00400024.

However, Jump = 1 causes the Jump MUX to select the Jump Target.

Therefore, Next PC = 0x00400080.

Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast
- We will improve performance by

Section 3-5 Quick Questions

Q1. For sub $t0, $t1, $t2 with $t1 = 20 and $t2 = 8, determine RegDst, ALUSrc, MemtoReg, RegWrite, ALUOp, ALU Control, and the final $t0.

Q2. For lw $t0, 8($t1), which control signals must be set to 1 to calculate the address, read memory, and write the loaded value to $t0?

Q3. Given RegWrite = 0, MemRead = 0, MemWrite = 1, ALUSrc = 1, and ALUOp = 00, identify the instruction type and the role of the ALU result.

Q4. For beq $t1, $t2, 12 with $t1 = $t2 = 25, PC = 0x00400020, and Branch = 1, determine Zero, PCSrc, and Next PC.

Q5. For a jump with Jump = 1, Branch = 0, Zero = 0, PC + 4 = 0x00400024, and Jump Target = 0x00400080, determine PCSrc and Next PC.

Exercise 1 — Control Signal Completion

Complete the missing control signal values for the following MIPS instructions. Use 0, 1, or X (Don't Care) where appropriate.

Control Signal	lw $t0, 8($t1)	sw $t2, 8($t1)	beq $t1, $t2,12
RegDst	0	?	?
ALUSrc	?	1	?
MemtoReg	?	?	?
RegWrite	1	?	0
MemWrite	0	?	0
ALUOp	?	00	?

Exercise 1 — Control Signal Completion

Answer

Control Signal	lw $t0, 8($t1)	sw $t2, 8($t1)	beq $t1, $t2,12
RegDst	0	X	X
ALUSrc	1	1	0
MemtoReg	1	X	X
RegWrite	1	0	0
MemWrite	0	1	0
ALUOp	00	00	01

Key Point: Control signals depend on the instruction type. Use X (Don't Care) when a signal does not affect instruction execution.

Exercise 2 — Control Signal Fault Analysis

Given lw $t0, 8($t1), with $t0 = 4, $t1 = 1000, Mem[1004] = 40, and Mem[1008] = 25.

Determine the final value of $t0 for each independent control signal error:

ALUSrc = 0

(b) RegWrite = 0

(c) MemtoReg = 0

Answe

r

(a) Address = 1000 + 4 = 1004 → $t0 = 40

(b) (b) Register write disabled → $t0 = 4

(c) (c) ALU result selected → $t0 = 1008

Key Point: Incorrect control signals affect address calculation, register writing, and write-back selection.

Exercise 3 — Reverse Control Signal Tracing

A MIPS processor performs the following operations:

ALU calculates 1000 + 12 = 1012.

Mem[1012] is updated to 35.

No register is modified.

Determine the values of ALUSrc, MemWrite, RegWrite, and MemtoReg.

Answer

ALUSrc = 1 (select immediate)

MemWrite = 1 (enable memory write)

RegWrite = 0 (disable register write)

MemtoReg = X (Don't Care)

Key Point: Store instructions use the ALU to calculate memory addresses and MemWrite to update memory, without register write-back.

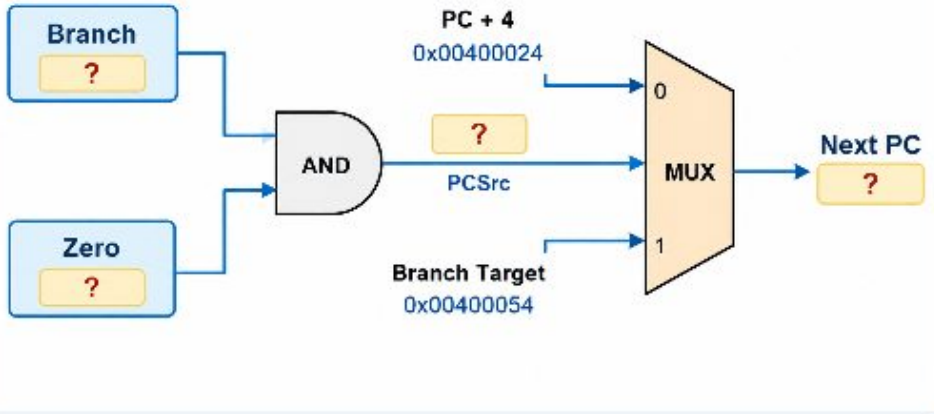
Exercise 4 — Reverse Branch Control Analysis

Use Branch Control diagram

Determine all possible combinations of Branch, Zero, and PCSrc for each case.

(a) Next PC = 0x00400054

(b) Next PC = 0x00400024



Answe

r

(a) PCSrc = 1. The only possible combination i

(b) Branch = 1, Zero = 1, PCSrc = 1.

(c) (b) PCSrc = 0. There are three possible combinations:

Branch	Zero	PCSrc
0	0	0
0	1	0
1	0	0

Key Point: PCSrc = Branch AND Zero. A branch is taken only when both signals are 1.

Exercise 5 — Jump Target Reconstruction

A MIPS processor executes j 0x00400080 with PC = 0x00400020. Given Jump = 1 and PCSrc = 0, determine:

(a) The 26-bit jump address field.

(b) The 32-bit Jump Target formed by concatenation.

(c) The final Next PC.

Answe

(a) Jump address field = 0x00100020.

(b) Jump Target = {(PC + 4)[31:28], Instruction[25:0], 2'b00} = 0x00400080.

(c) Jump = 1 selects the Jump Target. Therefore, Next PC = 0x00400080.

Key Point: A jump target combines the upper 4 bits of PC + 4, the 26-bit address field, and two trailing zeros.